

MÉMOIRE DE MASTER 1

Conception et développement d'une application web de suivi de la Coupe du Monde FIFA 2026

Par : CHANG Toma
Formation : Master 1 LDFS
Année universitaire : 2025–2026
Établissement : Ensitech

Date de soutenance : 2/06/26

Résumé

Ce mémoire présente la conception et le développement d'une application web dédiée au suivi de la Coupe du Monde FIFA 2026. L'objectif est de proposer une solution permettant de consulter les matchs, suivre les scores, visualiser les classements et observer l'évolution de la compétition en temps quasi réel.

Le projet s'inscrit dans une démarche complète : analyse des besoins, modélisation des données (Merise), conception UML, architecture technique, implémentation et validation fonctionnelle. L'application repose sur une stack moderne (Next.js, TypeScript, PostgreSQL) et intègre une API REST externe dédiée à la compétition 2026.

Une architecture hybride a été retenue : la base de données PostgreSQL sert de source principale pour le catalogue (listes, filtres, classements), tandis que l'API externe est utilisée pour les données live. Une synchronisation périodique alimente la base afin de limiter les appels externes et d'améliorer la robustesse de l'application.

Les tests réalisés confirment la conformité aux fonctionnalités attendues du cahier des charges. Le mémoire conclut sur les apports du projet, ses limites et les perspectives d'évolution.

Mots-clés : Coupe du Monde 2026, application web, API REST, PostgreSQL, Next.js, architecture hybride, temps quasi réel.

Table des matières

1. Introduction générale
2. Contexte et problématique
3. Analyse des besoins
4. État de l'art et choix technologiques
5. Conception du système
6. Architecture et implémentation
7. Intégration API et persistance des données
8. Réalisation des fonctionnalités
9. Tests, validation et résultats
10. Discussion, limites et perspectives
11. Conclusion générale
12. Bibliographie
13. Annexes

1. Introduction générale

La Coupe du Monde FIFA constitue l'un des événements sportifs les plus suivis au monde. Pour l'édition 2026, la compétition se déroule aux États-Unis, au Mexique et au Canada, avec un format élargi à 48 équipes. Cette évolution accroît la complexité du suivi des matchs, des classements et des phases éliminatoires.

Dans ce contexte, le présent mémoire traite de la conception et du développement d'une application web permettant de centraliser l'accès aux informations essentielles de la compétition. Le projet répond à un double objectif académique et technique :

démontrer la maîtrise d'un cycle complet de développement logiciel ;
mettre en œuvre une architecture intégrant API externe, base de données et interface utilisateur.

Le périmètre retenu couvre les fonctionnalités principales de consultation et de suivi, sans intégrer de paris, de comptes utilisateurs ou de fonctionnalités éditoriales avancées.

2. Contexte et problématique

2.1 Contexte du projet

Le sujet impose de concevoir une application de suivi sportif s'appuyant sur une API externe. Les données doivent être exploitées de manière fiable, lisible et exploitable par un utilisateur non expert.

Les contraintes majeures identifiées sont :

- disponibilité des données de la compétition 2026 ;
- fiabilité des sources externes ;
- performance de l'application ;
- séparation entre données catalogue et données live ;
- conformité aux bonnes pratiques de sécurité (pas d'exposition de secrets côté client).

2.2 Problématique

Comment concevoir une application web de suivi de la Coupe du Monde 2026 qui combine efficacement une API REST externe et une base de données locale, tout en garantissant une expérience utilisateur fluide et une mise à jour quasi temps réel des scores ?

2.3 Objectifs

Objectif général

Développer une application web fonctionnelle, responsive et maintenable pour le suivi de la CDM 2026.

Objectifs spécifiques

- Modéliser les données métier (Merise + UML).
- Intégrer une API REST externe dédiée à la compétition 2026.
- Persister les données catalogue en PostgreSQL.
- Implémenter les fonctionnalités F01 à F08 du cahier des charges.
- Valider le comportement de l'application par des tests manuels et techniques.

3. Analyse des besoins

3.1 Besoins fonctionnels

Réf.	Besoin	Description
F01	Consultation par phase	Filtrer les matchs par phase de compétition
F02	Liste des matchs	Afficher équipes, date/heure, score, statut
F03	Détail d'un match	Page dédiée avec informations complètes
F04	Mise à jour live	Rafraîchissement automatique des scores
F05	Classements	Tableaux par groupe
F06	Tableau éliminatoire	Visualisation des phases finales
F07	Navigation	Parcours simple entre sections
F08	Ergonomie	FR, responsive, chargement, erreurs

3.2 Besoins non fonctionnels

- Performance : affichage rapide des listes et classements via BDD.
- Disponibilité : continuité de service même en cas d'indisponibilité API live.
- Sécurité : appels API externes côté serveur uniquement.
- Maintenabilité : séparation services / UI / persistance.
- Accessibilité d'usage : interface claire, hiérarchie visuelle des scores et statuts.

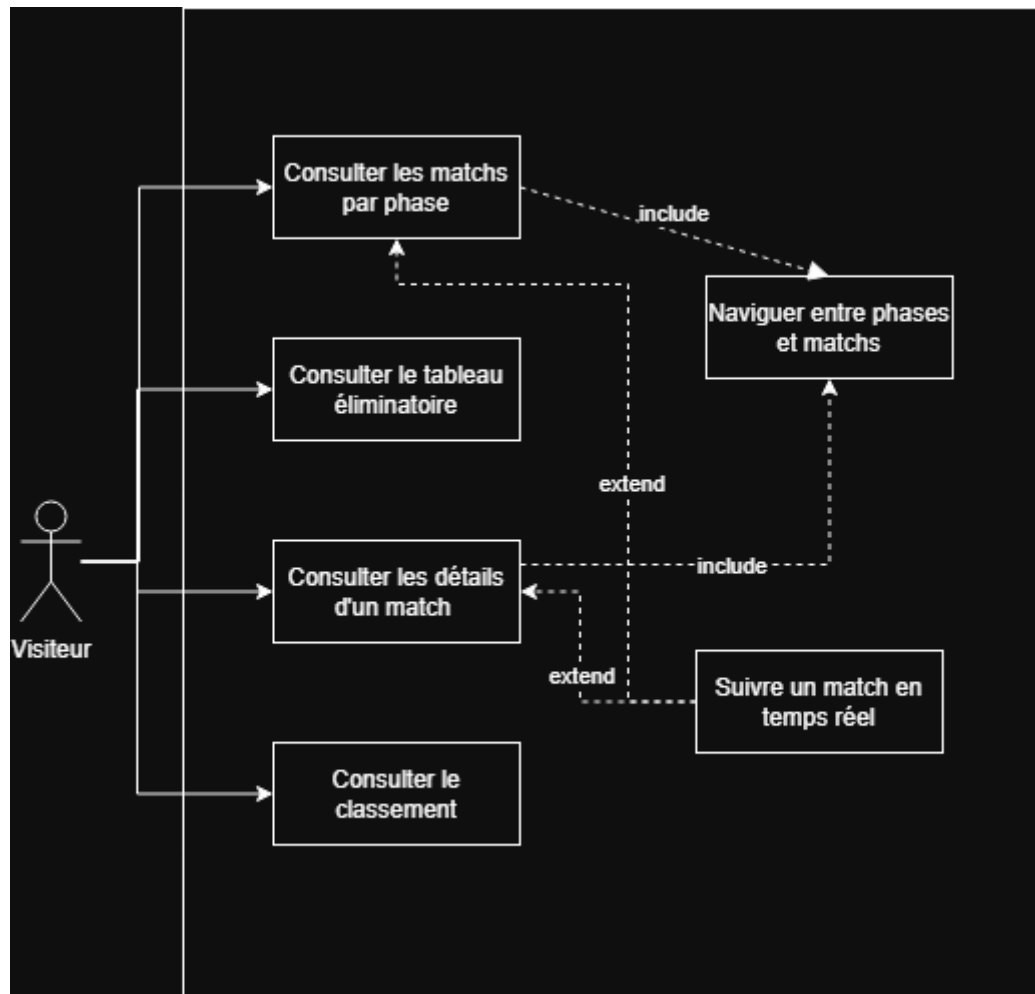
3.3 Acteurs et cas d'utilisation

Acteur principal : utilisateur occasionnel ou régulier souhaitant consulter les résultats et le live.

Cas d'utilisation principaux :

- consulter les matchs d'une phase ;
- ouvrir le détail d'un match ;
- suivre les matchs en direct ;
- consulter les classements ;
- visualiser le tableau éliminatoire.

Figure A – Cas d'utilisation



4. État de l'art et choix technologiques

4.1 Solutions existantes

Des plateformes comme FlashScore, Sofascore ou les sites officiels proposent déjà un suivi sportif avancé. Cependant, le projet académique vise à construire une solution maîtrisée de bout en bout, avec intégration API, modélisation et persistance.

4.2 Choix de l'API externe

Plusieurs sources ont été étudiées :

- API-Football (quota/limitations sur la saison 2026 en plan gratuit) ;
- sources JSON open data ;
- API communautaire worldcup26.ir.

Choix retenu : worldcup26.ir

Motifs :

- API REST documentée (Swagger) ;
- données CDM 2026 ;
- endpoints adaptés (games, groups, teams, stadiums) ;
- utilisation possible sans clé pour la lecture publique.

4.3 Stack technique retenue

Couche	Technologie	Justification
Front	Next.js + React + TypeScript	SSR, routing, typage
Back applicatif	API Routes Next.js	proxy et logique serveur
Données	PostgreSQL	persistance relationnelle
Conteneurisation	Docker Compose	environnement reproductible
Style	Tailwind CSS	UI responsive rapide

5. Conception du système

5.1 Modélisation Merise

Le modèle conceptuel, logique et physique structure les entités métier du tournoi.

Figures :

Figure B – MCD :

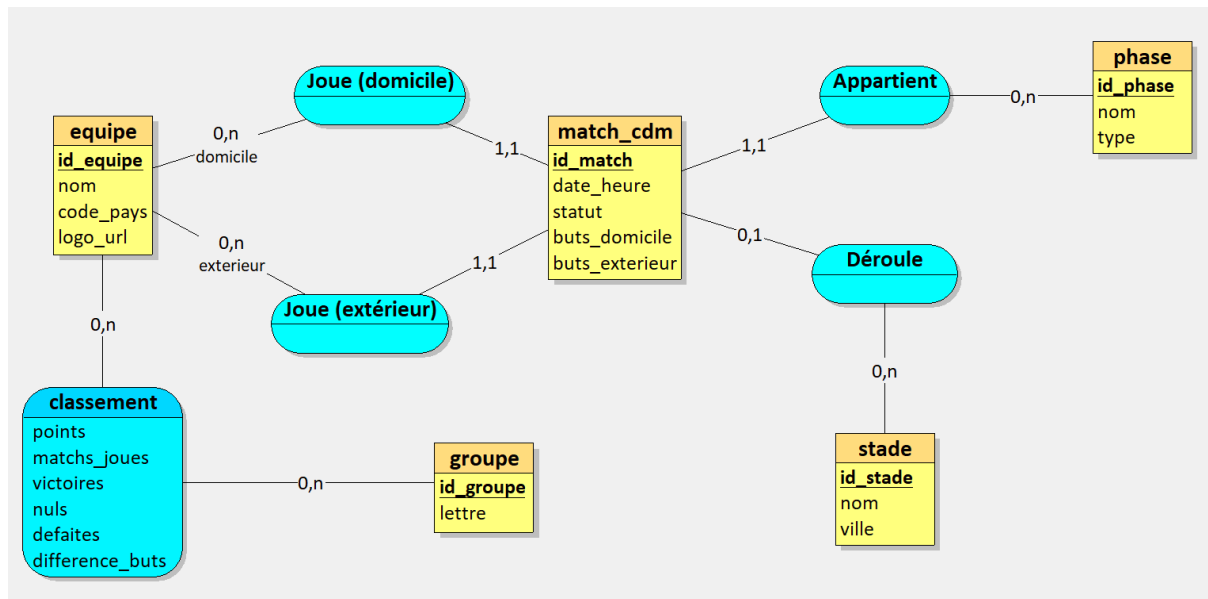


Figure C – MLD :

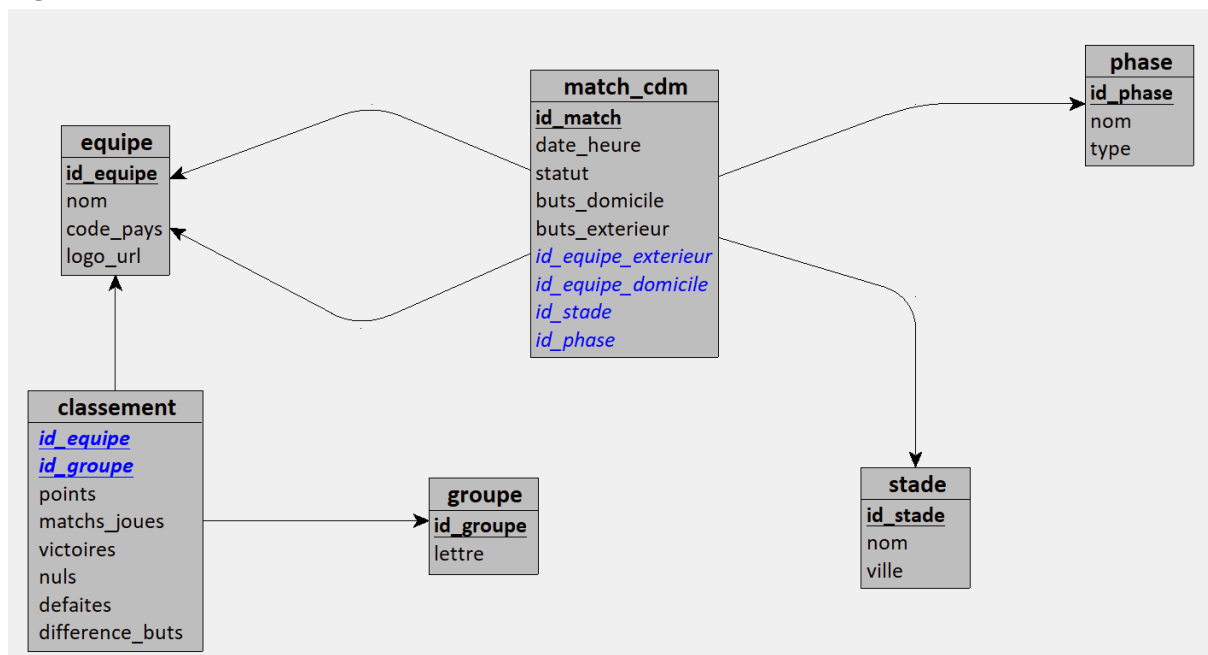
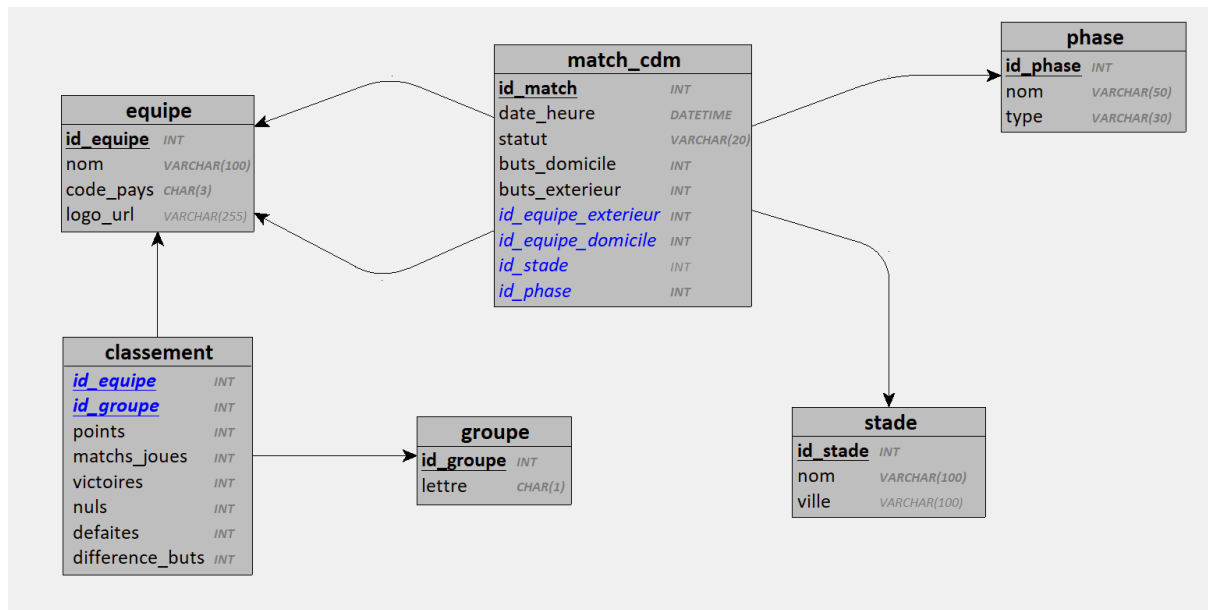


Figure D – MPD :



Tables principales :

equipe, phase, stade, groupe, match_cdm, classement, sync_meta.

Le dictionnaire de données détaillé est disponible dans :

1. Vue d'ensemble

Table	Description	Clé primaire
equipe	Sélection nationale participant à la compétition	id_equipe
phase	Étape du tournoi (poules, huitièmes, finale...)	id_phase
stade	Lieu d'accueil d'un match	id_stade
groupe	Poule de la phase de groupes	id_groupe
match_cdm	Rencontre opposant deux équipes	id_match
classement	Statistiques d'une équipe dans un groupe (association CLASSE_DANS au MCD)	(id_equipe , id_groupe)

2. Table `equipe`

Description : Représente une équipe nationale.

Attribut	Type	Taille	Oblig.	Clé	Description
<code>id_equipe</code>	ENTIER	—	Oui	PK	Identifiant unique de l'équipe
<code>nom</code>	CHAÎNE	100	Oui	—	Nom officiel de la sélection (ex. « France »)
<code>code_pays</code>	CHAÎNE	3	Oui	—	Code pays FIFA sur 3 caractères (ex. « FRA »)
<code>logo_url</code>	CHAÎNE	255	Non	—	URL du drapeau ou du logo de l'équipe

3. Table `phase`

Description : Découpe le tournoi en étapes successives.

Attribut	Type	Taille	Oblig.	Clé	Description
<code>id_phase</code>	ENTIER	—	Oui	PK	Identifiant unique de la phase
<code>nom</code>	CHAÎNE	50	Oui	—	Libellé affiché (ex. « Phase de groupes », « Finale »)
<code>type</code>	CHAÎNE	30	Oui	—	Code de la phase : <code>groupes</code> , <code>seizieme</code> , <code>huitieme</code> , <code>quart</code> , <code>demi</code> , <code>petite_finale</code> , <code>finale</code>

4. Table **stade**

Description : Stade où peut se dérouler un match.

Attribut	Type	Taille	Oblig.	Clé	Description
id_stade	ENTIER	—	Oui	PK	Identifiant unique du stade
nom	CHAÎNE	100	Oui	—	Nom du stade (ex. « MetLife Stadium »)
ville	CHAÎNE	100	Non	—	Ville d'implantation du stade

5. Table **groupe**

Description : Poule de la phase de groupes (lettre A à L en CDM 2026).

Attribut	Type	Taille	Oblig.	Clé	Description
id_groupe	ENTIER	—	Oui	PK	Identifiant unique du groupe
lettre	CHAÎNE	1	Oui	—	Lettre du groupe (ex. « A », « B »)

5.2 Conception UML

Les diagrammes UML documentent la structure et les interactions :

- diagramme de classes ;
- diagrammes de séquence (consultation, mise à jour score) ;
- diagrammes d'activité (navigation, temps réel).

Figure E – Classes :

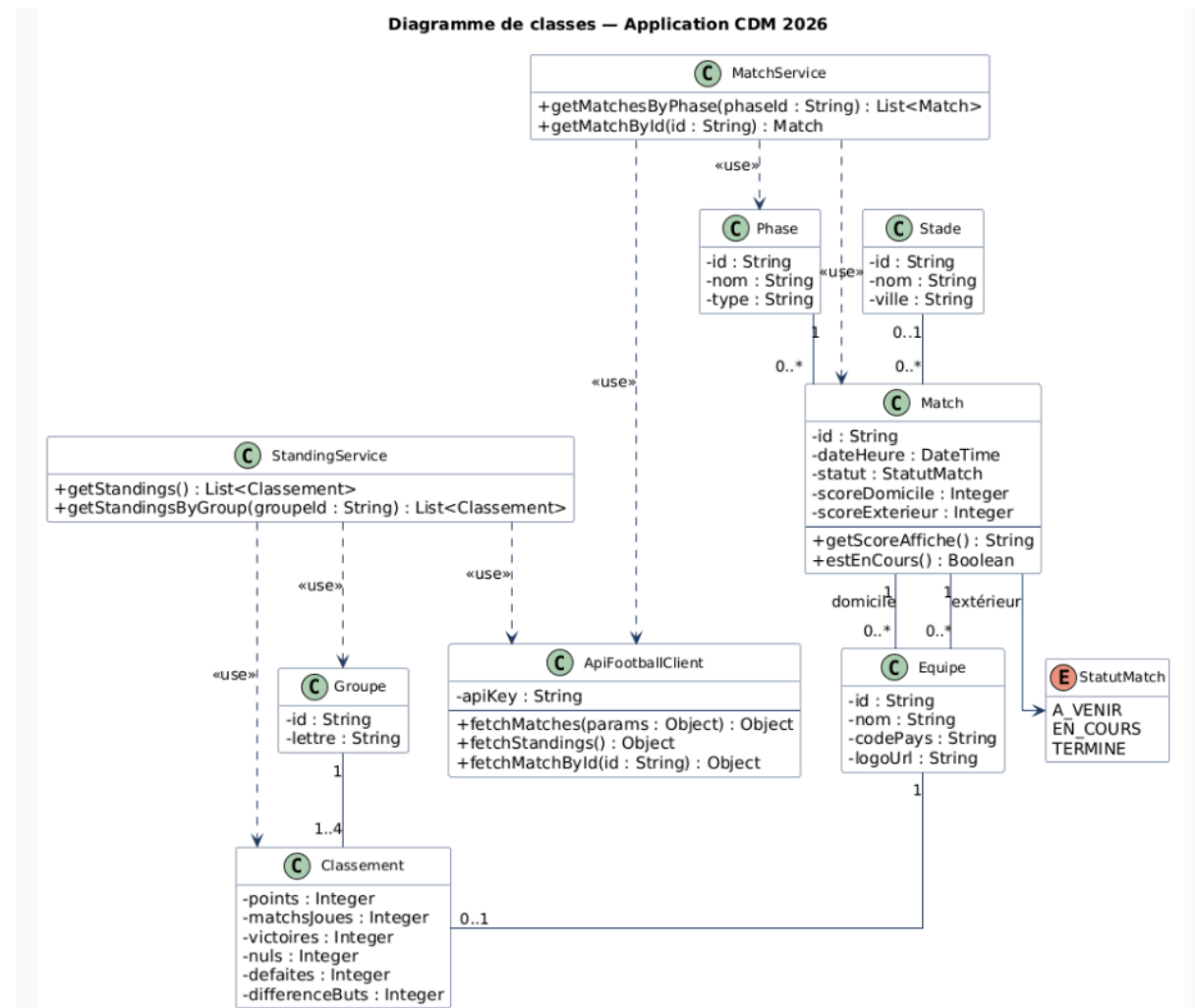


Figure F – Séquence consultation :

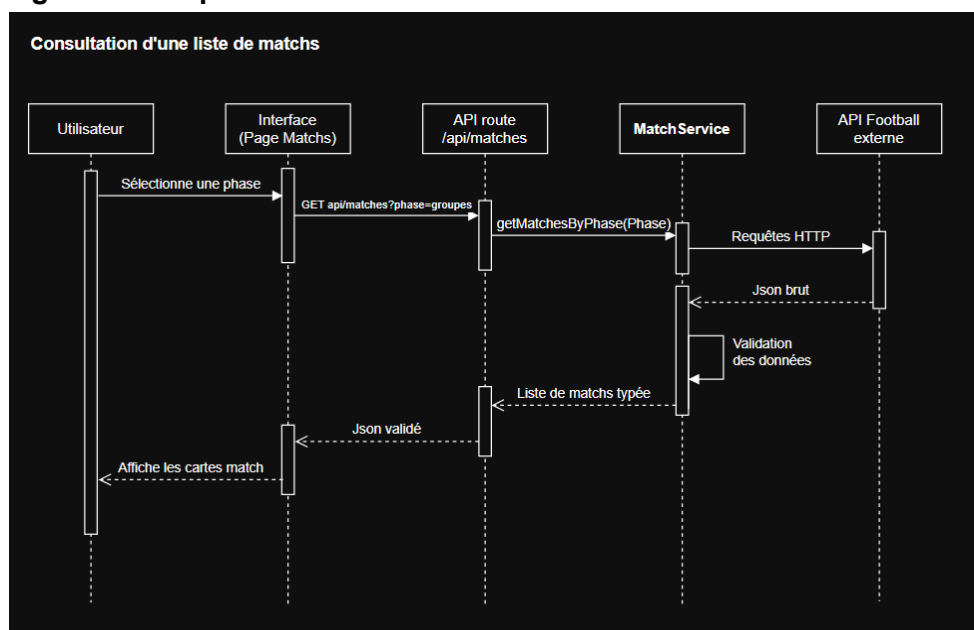


Figure G – Séquence score live :

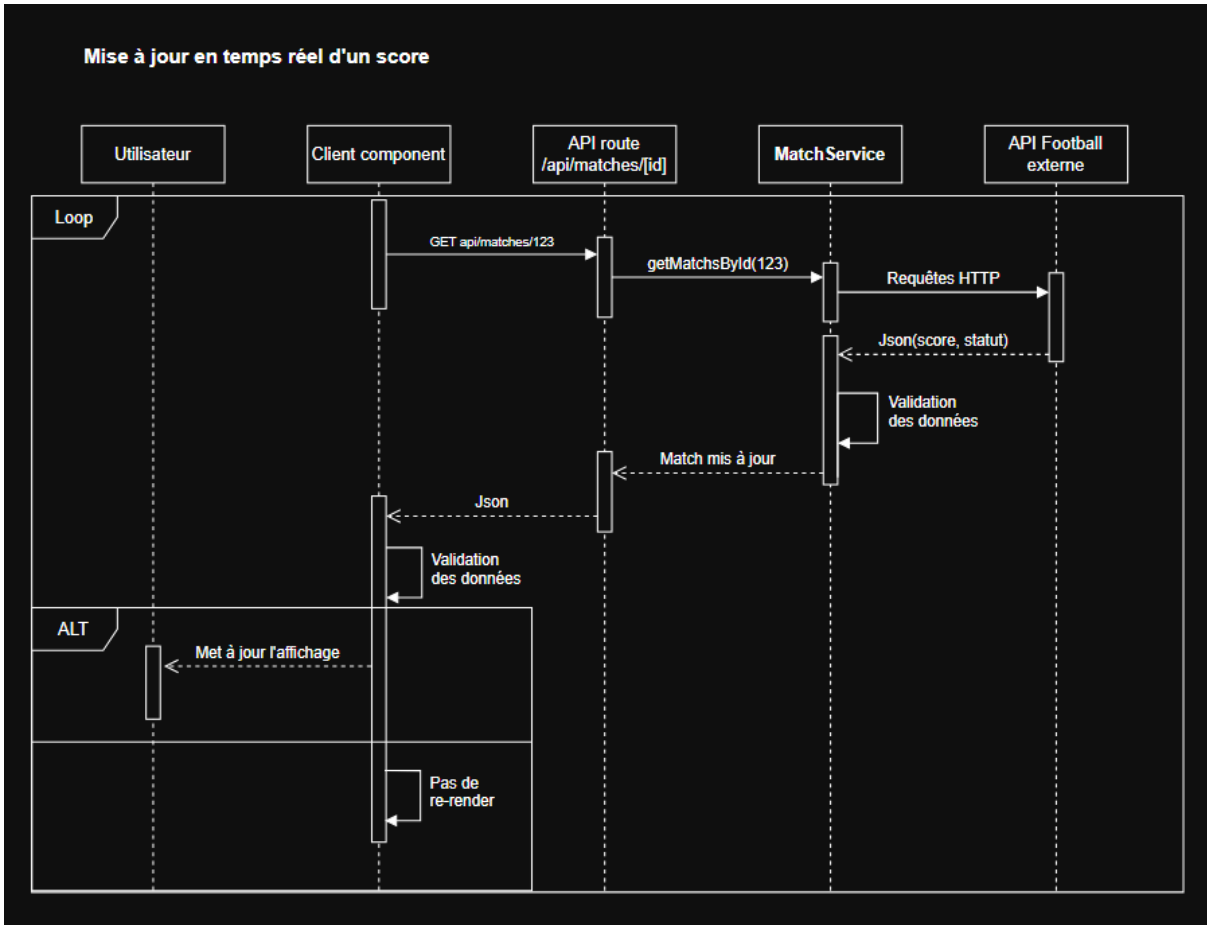


Figure H – Activité navigation :

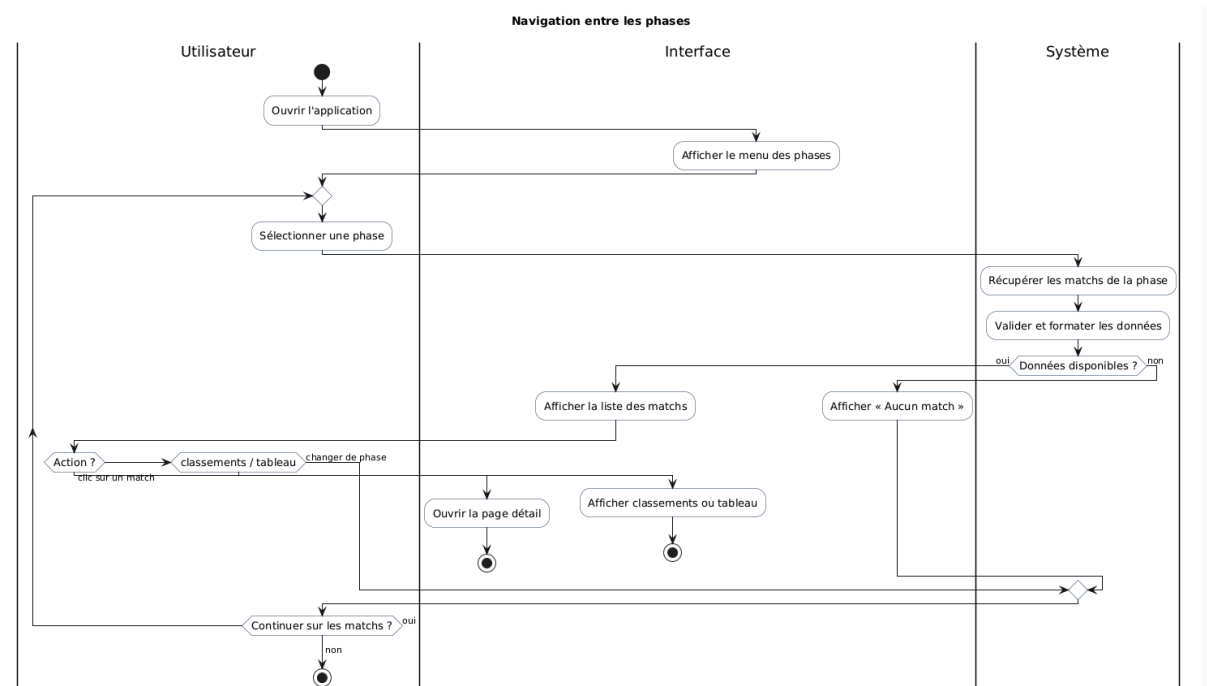
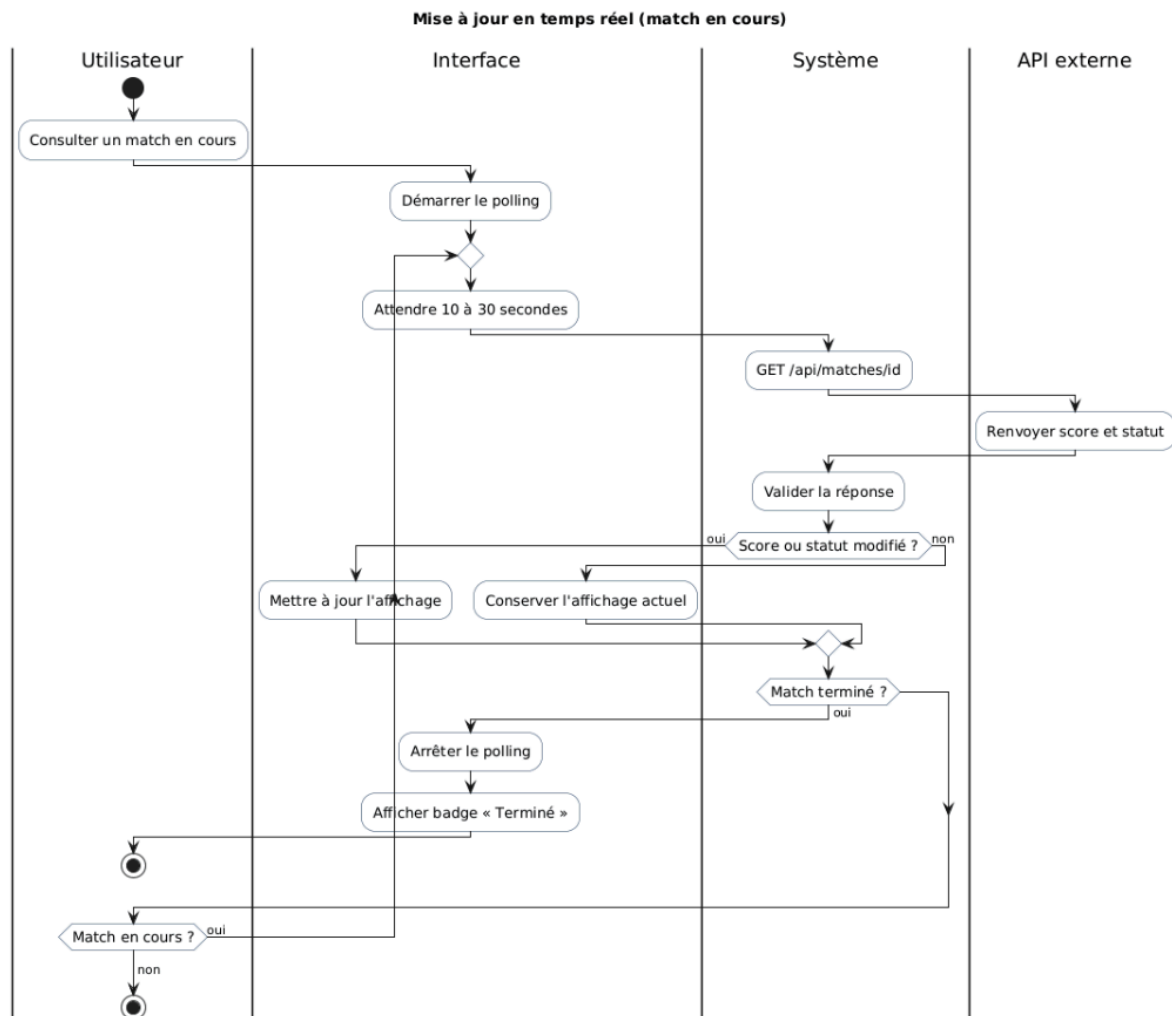


Figure I – Activité temps réel :



5.3 Conception IHM

La conception interface suit le cycle zoning → wireframe → maquette.

Figures :

Figure J – Zoning :

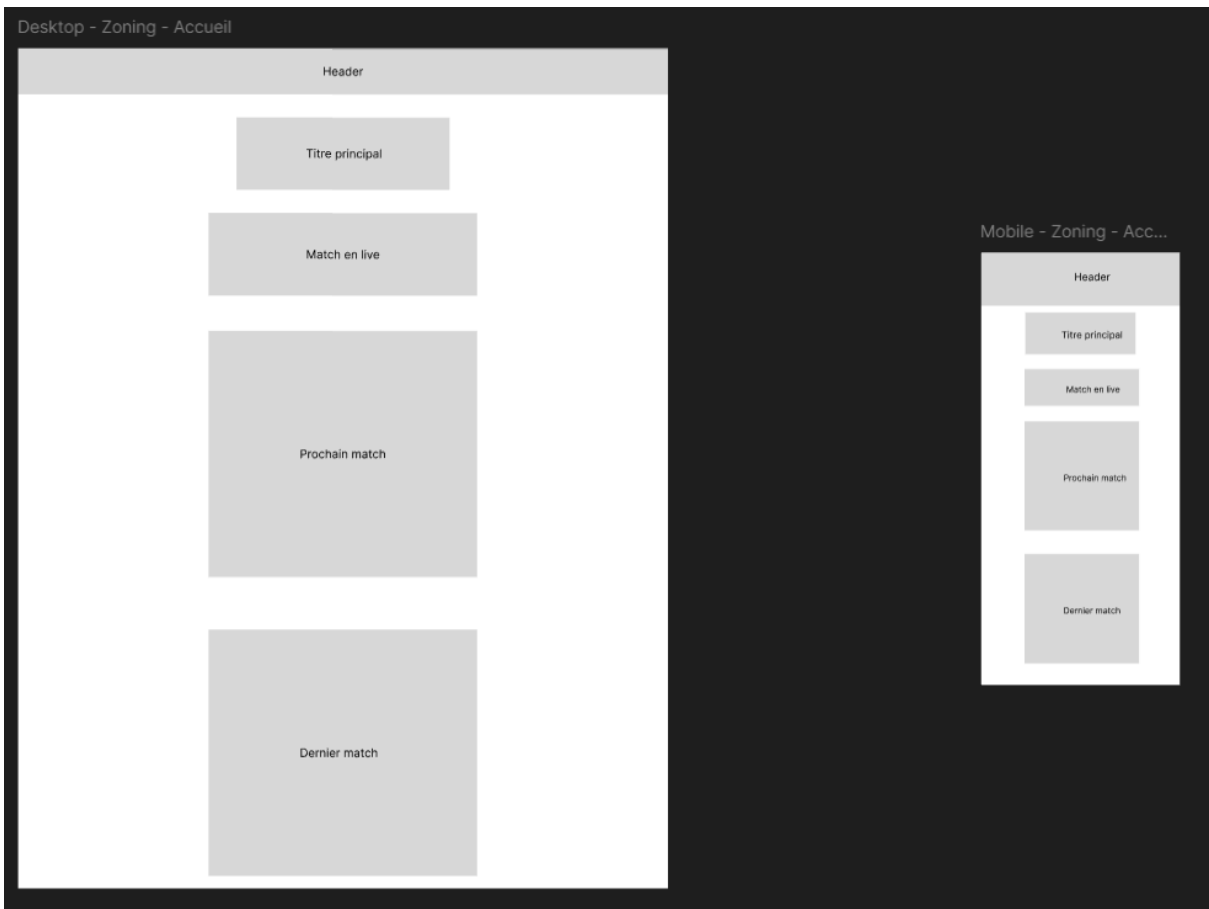


Figure K – Wireframe :

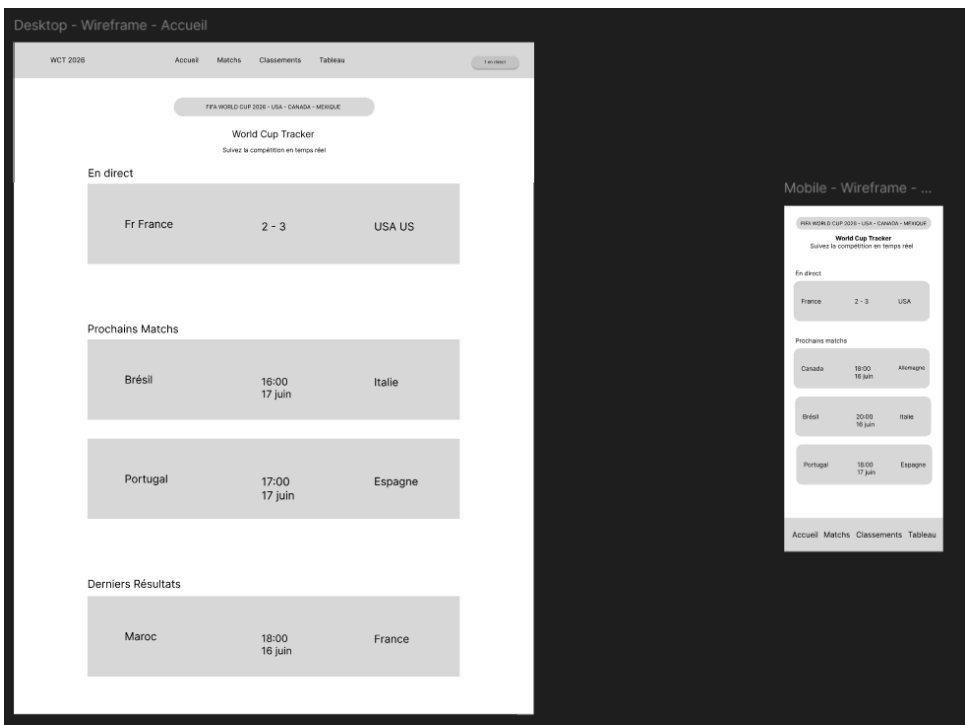
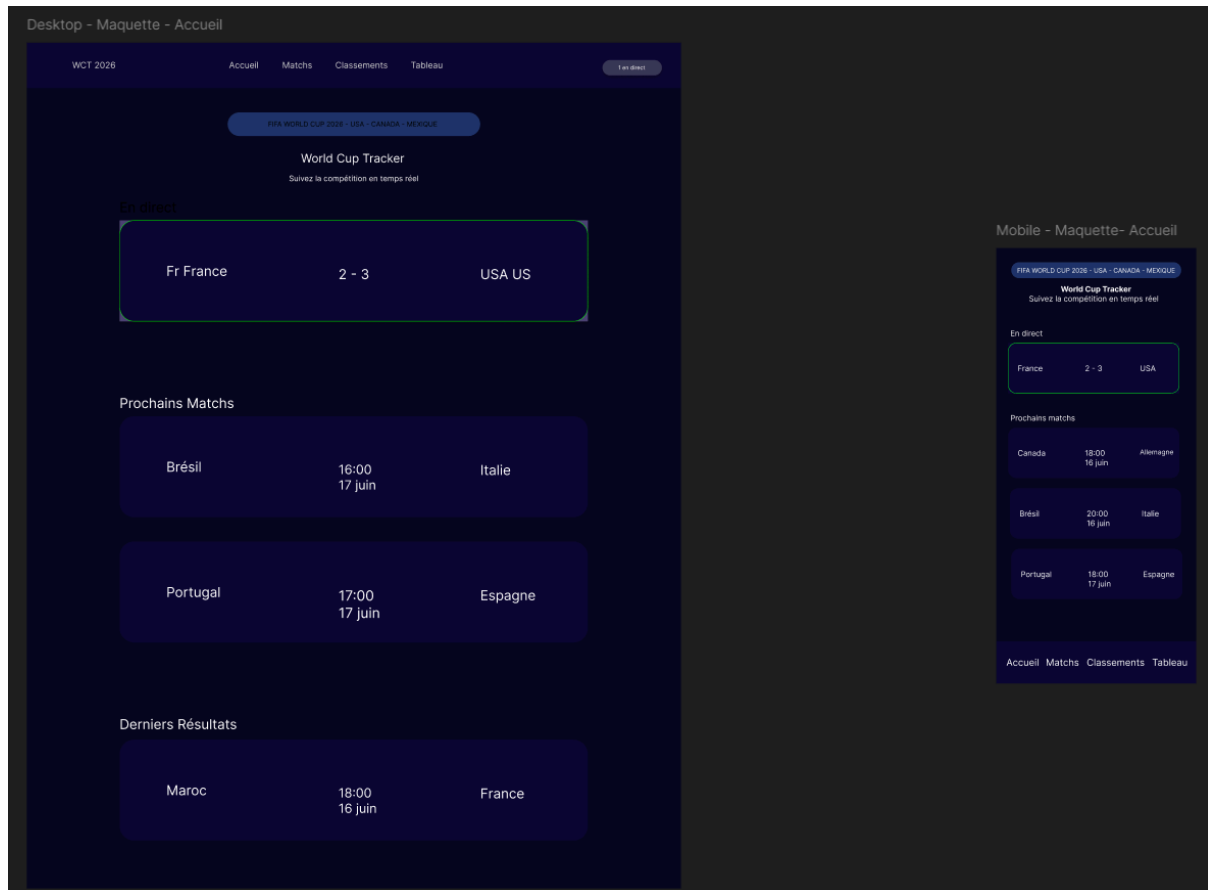


Figure L – Maquette :



6. Architecture et implémentation

6.1 Architecture globale

L'architecture retenue est hybride :

- Catalogue (listes, filtres, classements, tableau) → PostgreSQL
- Live (matches en cours) → API externe en direct
- Synchronisation → script périodique API → BDD

Ce choix répond à la problématique initiale : concilier fraîcheur des données live et performance/robustesse du catalogue.

6.2 Organisation du code

Structure principale :

src/app/ : pages et routes API ;
src/components/ : composants UI ;
src/lib/api/ : services métier ;
src/lib/db/ : accès PostgreSQL ;
src/lib/sync/ : orchestration de synchronisation ;
scripts/sync-worldcup2026.mjs : import catalogue.

6.3 Sécurité et configuration

Les variables sensibles sont gérées via `.env.local` côté serveur.

Le front n'appelle jamais directement l'API externe : il passe par les routes internes (`/api/matches`, `/api/standings`, etc.).

7. Intégration API et persistance des données

7.1 Endpoints exploités

GET /get/games : matchs et scores ;
GET /get/groups : classements ;
GET /get/teams : équipes ;
GET /get/stadiums : stades.

Documentation : <https://worldcup26.ir/api-docs>

7.2 Mapping API → base

Exemples de correspondance :

id API → `match_cdm.id_match`
`home_team_id` / `away_team_id` → clés étrangères équipes
`home_score` / `away_score` → `buts_domicile` / `buts_exterieur`
`type (group, r32, r16, etc.)` → `phase.type`
données groupes API → table classement

7.3 Synchronisation

Commande :

npm run sync:wc2026

Résultat observé lors des tests :

- 73 matchs terminés
- 31 matchs à venir
- 48 lignes de classement

La table sync_meta conserve la date de dernière synchronisation et les compteurs.

7.4 Politique de repli (fallback)

- Catalogue : continue via BDD si API indisponible.
- Live : dépend de l'API ; en cas d'échec, pas de score live actualisé.
- Détail match : retour possible sur la dernière version BDD.

8. Réalisation des fonctionnalités

8.1 Pages principales

- / : accueil (live + prochains + derniers résultats)
- /matches : liste filtrable par phase
- /matches/[id] : détail match
- /live : tableau live
- /standings : classements
- /bracket : tableau éliminatoire

8.2 Validation fonctionnelle (F01–F08)

8.2 Validation fonctionnelle (F01–F08)		
Fonctionnalité	Statut	Preuve
F01 phases	Réalisé	capture page matchs
F02 liste	Réalisé	capture liste
F03 détail	Réalisé	capture détail
F04 live auto	Réalisé	polling 30s
F05 classements	Réalisé	capture standings
F06 tableau	Réalisé	capture bracket
F07 navigation	Réalisé	menu + liens
F08 ergonomie	Réalisé	loading/erreurs/responsive

8.3 Captures de la version finale

Figure M – Accueil

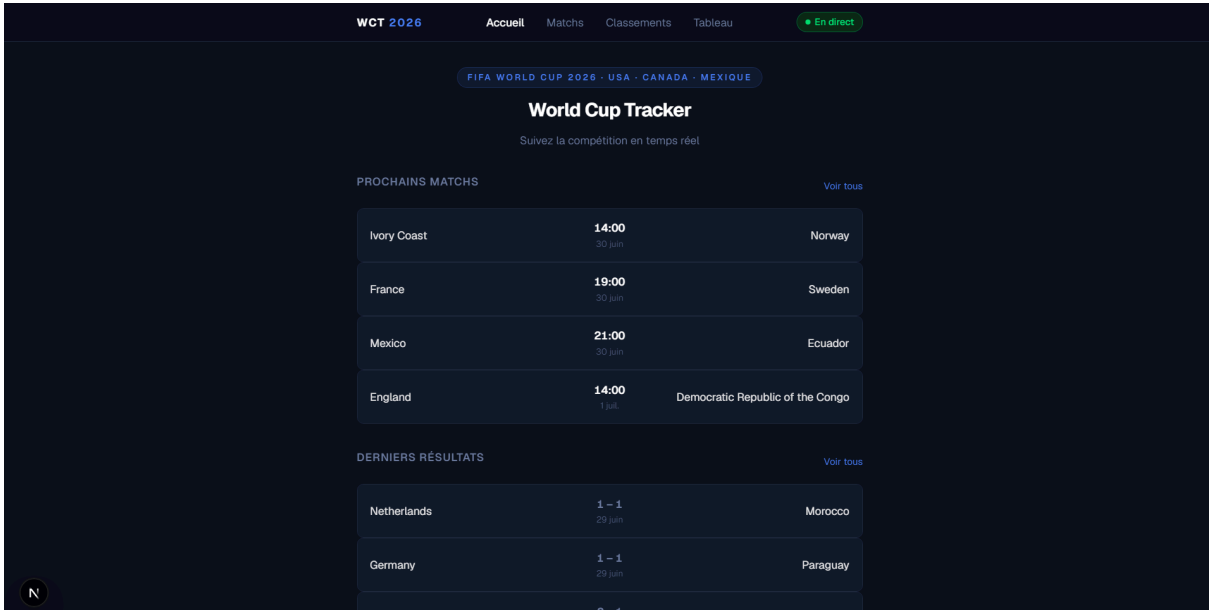


Figure N – Matches par phase

WCT 2026									
Accueil									
Classements									
En direct									
Classements — Phase de groupes									
GROUPE A					GROUPE B				
MJ V N D DB Pts					MJ V N D DB Pts				
1	MX	Me...	3	3	0	0	+6	9	
2	ZA	Sou...	3	1	1	1	-1	4	
3	KR	South...	3	1	0	2	-1	3	
4	CZ	Czec...	3	0	1	2	-4	1	
GROUPE C					GROUPE D				
MJ V N D DB Pts					MJ V N D DB Pts				
1	BR	Bra...	3	2	1	0	+6	7	
2	MA	Mo...	3	2	1	0	+3	7	
3	SC	Scott...	3	1	0	2	-3	3	
4	HT	Haiti	3	0	0	3	-6	0	
GROUPE E					GROUPE F				
MJ V N D DB Pts					MJ V N D DB Pts				
1	DE	Ger...	3	2	0	1	+6	6	
2	CI	Ivor...	3	2	0	1	+2	6	
3	EC	Ecuad...	3	1	1	1	0	4	
4	CW	Cura...	3	0	1	2	-8	1	
GROUPE F					GROUPE F				
MJ V N D DB Pts					MJ V N D DB Pts				
1	NL	Net...	3	2	1	0	+6	7	
2	JP	Jap...	3	1	2	0	+4	5	
3	SE	Swed...	3	1	1	1	0	4	
4	TN	Tunisia	3	0	0	3	-10	0	

Figure O – Détail match

WCT 2026

Accueil

Matches

Classements

Tableau

En direct

Retour

Groupe A - Phase de groupes

MX

Mexico

2 - 0

Terminé

ZA

South Africa

Date et heure

11 juin 2026 à 15:00

Phase

Phase de groupes

Groupe

Groupe A

Stade

Estadio Azteca - Mexico City

N

Figure P – Page live

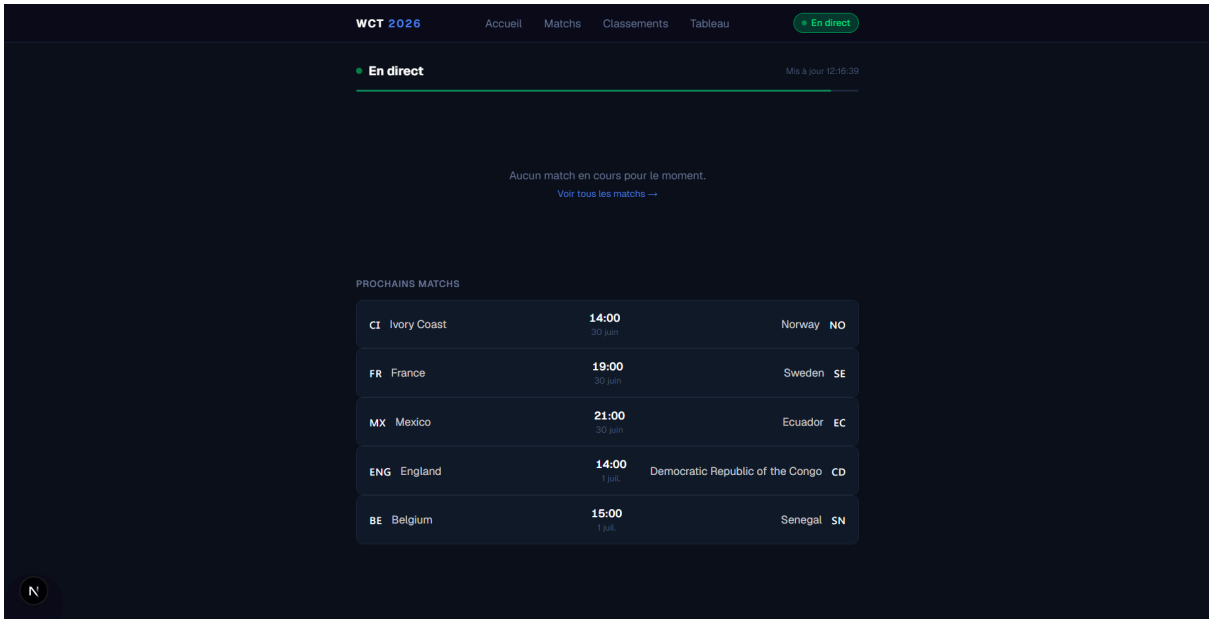


Figure Q – Classements



Figure R – Tableau éliminatoire

SEIZIÈMES DE FINALE			HUITIÈMES DE FINALE		QUARTS DE FINALE	
ZA South Africa	0		CA Canada		UN Winner Match 89	
CA Canada	1		MA Morocco		UN Winner Match 90	
Terminé			4 juil. - 14:00		9 juil. - 18:00	
BR Brazil	2		PY Paraguay		UN Winner Match 93	
JP Japan	1		UN Winner Match 77		UN Winner Match 94	
Terminé			4 juil. - 19:00		10 juil. - 14:00	
DE Germany	1		BR Brazil		UN Winner Match 91	
PY Paraguay	1		UN Winner Match 78		UN Winner Match 92	
Terminé			5 juil. - 18:00		11 juil. - 19:00	
NL Netherlands	1		UN Winner Match 79		UN Winner Match 95	
MA Morocco	1		UN Winner Match 80		UN Winner Match 96	
Terminé			5 juil. - 20:00		11 juil. - 22:00	
CI Ivory Coast			UN Winner Match 83			
NO Norway			UN Winner Match 84			
30 juil. - 14:00			6 juil. - 16:00			
FR France			UN Winner Match 81			

Figure S – Swagger API

```
GET /bracket 200 in 888ms
GET /api/matches?live=true 200 in 266ms
GET /api/matches?live=true 200 in 402ms
GET / 200 in 20ms
GET /api/matches 200 in 61ms
GET /api/matches 200 in 21ms
GET /api/matches?live=true 200 in 273ms
GET /api/matches?live=true 200 in 101ms
GET /bracket 200 in 105ms
```

Figure T – Terminal sync réussie

```
GET /api/matches?live=true 200 in 105ms
[sync] Catalogue importé en base.
GET /api/matches 200 in 3960ms
[sync] Catalogue importé en base.
```

9. Tests, validation et résultats

9.1 Protocole de test

Tests réalisés :

- Démarrage environnement (docker compose up -d, npm run dev)
- Synchronisation catalogue (npm run sync:wc2026)
- Vérification des pages principales
- Vérification des routes API internes

- Vérification compilation TypeScript

9.2 Résultats

- Compilation TypeScript : OK
- Synchronisation BDD : OK
- Affichage catalogue : OK
- Navigation inter-pages : OK
- Mise à jour live : OK en quasi temps réel (30 s)

9.3 Analyse critique

Points forts :

- architecture claire et justifiable ;
- séparation catalogue/live pertinente ;
- conformité au cahier des charges ;
- base de conception solide (Merise + UML).

Points perfectibles :

- dépendance à une API tierce ;
- absence de WebSocket ;
- fonctionnalités optionnelles non implémentées (timeline buts, filtre équipe avancé).

10. Discussion, limites et perspectives

10.1 Limites

- Le live n'est pas instantané (polling, pas push temps réel).
- La qualité des données dépend de la source externe.
- Le fallback live via BDD est limité (catalogue non stocké en en_cours).
-

10.2 Perspectives

- Ajouter une timeline des buts (F09).
- Ajouter un filtre par équipe (F10).
- Ajouter un mode sombre/clair commutable (F11).
- Mettre en place des tests automatisés (Jest/Playwright).
- Préparer un déploiement cloud (Vercel + base managée).

11. Conclusion générale

Ce mémoire a permis de concevoir et développer une application web complète de suivi de la Coupe du Monde 2026.

Le travail réalisé couvre l'ensemble de la chaîne projet : analyse, modélisation, architecture, implémentation, intégration API, persistance et validation.

L'architecture hybride retenue démontre une approche pragmatique : la base de données assure la stabilité du catalogue, tandis que l'API externe garantit la fraîcheur des données live. Cette solution répond à la problématique initiale et aux exigences du sujet.

Le projet constitue une base solide, évolutive et présentable en soutenance, avec des axes d'amélioration clairement identifiés.

Bibliographie / Webographie

1. FIFA World Cup 2026 – informations générales : <https://www.fifa.com>
2. API worldcup2026 (GitHub) : <https://github.com/rezarahiminia/worldcup2026>
3. Documentation API : <https://worldcup26.ir/api-docs>
4. Next.js Documentation : <https://nextjs.org/docs>
5. PostgreSQL Documentation : <https://www.postgresql.org/docs/>
6. React Documentation : <https://react.dev>
7. Tailwind CSS Documentation : <https://tailwindcss.com/docs>

Annexes

Annexe A – Commandes d'exécution

- docker compose up -d
- npm run sync:wc2026
- npm run dev

Annexe B – Variables d'environnement

- USE MOCK_DATA=false
- NEXT_PUBLIC_POLL_INTERVAL_MS=30000

- DATABASE_URL=postgresql://cdm:cdm2026@localhost:5432/cdm2026
- WORLDCUP2026_API_URL=<https://worldcup26.ir>

Annexe C – Routes API internes

- GET /api/matches
- GET /api/matches/[id]
- GET /api/standings
- GET /api/sync
- POST /api/sync

Annexe D – Extraits de structure SQL

Tables principales : equipe, phase, stade, groupe, match_cdm, classement, sync_meta.